JAVA 2

Module JUN

Tests unitaires avec JUnit

Au programme

1. À vous d'écrire les tests
2. Écrire un test
3. Vérifier un contrat
4. Choisir et justifier
5. La testabilité

1. À vous d'écrire les tests



Qui écrit quoi

Jusqu'ici, le test était fourni et vous écriviez le code. Dans ce module, les rôles s'inversent : c'est à vous d'écrire les tests.

Exercice	Fourni	À écrire
Jusqu'ici	le test	le code
`JUN.1`	le test	le code, une dernière fois
`JUN.2`	quatre tirelires	la vérification
`JUN.3`	la classe `Moyenne`	sa classe de test
Examen	une classe d'un tiers	sa classe de test

Un programme qui affiche ne vérifie rien

Le programme affiche, et c'est vous qui jugez

Il faut connaître la bonne réponse, et relire à chaque lancement

Un test connaît la réponse, et la vérifie seul

```
public static void main(String[] args)
        throws SalleCompleteException {
    Salle salle = new Salle(100);

    salle.reserver(2);

    System.out.println(salle.placesLibres());   // 98 ?
}
```

Démonstration

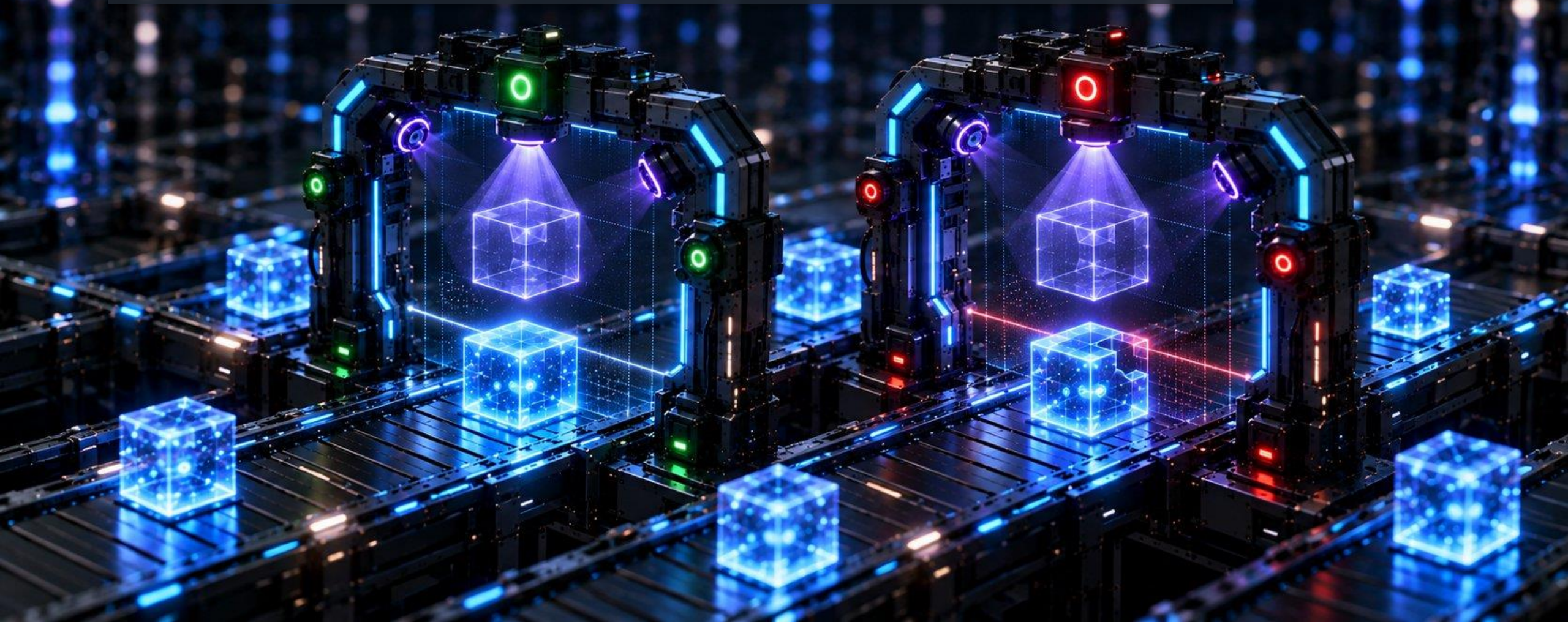


Démonstration

Rouge, vert, puis nettoyer

Déroulé : Demo_JUN

2. Ce qu'un test contient



Où vit une classe de test

Dans `src/test/java`, sous le même package que la classe testée

Son nom : celui de la classe, suivi de `Test`

`JUnit` n'est visible que de ce côté : `TEC-MVN.2`

```java
package be.ephec.java.spectacle;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class SalleTest {

    // une methode par comportement verifie
}
```

`@Test`, puis une assertion

`@Test` fait d'une méthode un test, que `JUnit` lance seul

`assertEquals` reçoit l'attendu d'abord, l'obtenu ensuite

Sans assertion, le test ne peut qu'être vert : `CC-TST.2`

Trois temps, séparés par une ligne vide :

- préparer,
- agir,
- vérifier

```java
@Test
void uneSalleNeuveEstEntierementLibre() {
    Salle salle = new Salle(100);

    int libres = salle.placesLibres();

    assertEquals(100, libres);
}
```

Les assertions dont vous aurez besoin

Assertion	Réussit si
`assertEquals(attendu, obtenu)`	les deux valeurs sont égales
`assertTrue(condition)`	la condition est vraie
`assertFalse(condition)`	la condition est fausse
`assertNull(valeur)`	la valeur est `null`
`assertThrows(Type.class, ...)`	l'appel lève cette exception

Un message qui explique l'échec

Le dernier argument : un message, facultatif pour `JUnit`

Dans vos tests : toujours

Il décrit l'attente, avec les mots du métier

Sur les slides, il est parfois omis pour la lisibilité

```
assertEquals(98, salle.placesLibres(),
        "Deux places reservees sur 100 doivent en laisser 98");

// en cas d'echec, JUnit affiche :
// Deux places reservees sur 100 doivent en laisser 98
//     ==> expected: <98> but was: <100>
```

Un test rouge accuse le code, ou le test

Une assertion fausse : JUnit affiche l'attendu et l'obtenu

Une exception imprévue : le test échoue sur sa trace

Avant de corriger quoi que ce soit, relire la spécification

```
// une assertion fausse
expected: <98> but was: <100>

// une exception que personne n'attendait
java.lang.UnsupportedOperationException:
    reserver n'est pas encore implementee
```

Exercices



JUN.1 Le test comme spécification

fascicule JUN

Lire `CalculatriceTest` avant d'écrire une ligne
Tests : `CC-TST.1`, `CC-TST.2`

3. Vérifier tout le contrat



Une documentation est une liste de tests

Chaque phrase de la documentation est une promesse

Chaque promesse mérite son test

Les promesses faites en cas d'échec aussi

```
/**
 * Reserve des places.
 *
 * @throws IllegalArgumentException si places <= 0
 * @throws SalleCompleteException   s'il reste moins de
 *         places que demande ; rien n'est alors reserve
 */
public void reserver(int places)
        throws SalleCompleteException
```

Vérifier qu'une exception est levée

`assertThrows` reçoit le type attendu et l'appel à surveiller

Le test échoue si rien n'est levé, ou autre chose

La flèche `->` est une lambda : recopiez la forme, LAM l'expliquera

```
@Test
void reserverZeroPlaceEstRefuse() {
    Salle salle = new Salle(100);

    assertThrows(IllegalArgumentException.class,
            () -> salle.reserver(0),
            "Reserver zero place doit etre refuse");
}
```

Le `throws` d'une méthode de test

Préparer ou agir peut appeler une méthode qui déclare une exception

La méthode de test la déclare à son tour

Jamais de `catch` dans un test : il le rendrait vert

```
@Test
void deuxPlacesReserveesNeSontPlusLibres()
        throws SalleCompleteException {
    Salle salle = new Salle(100);

    salle.reserver(2);

    assertEquals(98, salle.placesLibres(),
            "Deux places reservees sur 100 doivent en lais
}
```

Après l'échec, l'état

Lever la bonne exception ne suffit pas

Le refus ne doit rien avoir changé

Une assertion pour l'échec, une pour l'état

```java
@Test
void unePlaceDeTropNeReserveRien()
        throws SalleCompleteException {
    Salle salle = new Salle(10);
    salle.reserver(8);

    assertThrows(SalleCompleteException.class,
            () -> salle.reserver(3),
            "Une demande trop grande doit etre refusee");

    assertEquals(2, salle.placesLibres(),
            "Le refus ne doit rien reserver");
}
```

Exercices



JUN.2 Détecter un défaut

fascicule JUN

Ne lisez pas le code des tirelires : faites-les travailler

4. Choisir ses cas, et les justifier



Où se logent les défauts

On ne peut pas tout tester : on regroupe les cas en familles, on en teste un de chaque, et l'on vise les frontières.

Famille de cas	Pour une salle de 10 places
le cas ordinaire	réserver 3 places
la borne atteinte	réserver exactement les places libres
la borne dépassée	réserver une place de trop
la valeur interdite	réserver 0 place, ou -1
la répétition	deux réservations successives

Un cas, un test, un nom

Une méthode par comportement : CC-TST.2

Son nom dit ce qui est vérifié, jamais un numéro : CC-NOM.1

`@DisplayName` l'énonce en clair, avec des espaces

```
@Test
@DisplayName("Reserver toutes les places libres remplit la salle")
void reserverToutesLesPlacesLibres()
        throws SalleCompleteException {
    Salle salle = new Salle(10);

    salle.reserver(10);

    assertEquals(0, salle.placesLibres(),
            "Reserver les 10 places doit remplir la salle");
}
```

Une valeur, un test

Une valeur vérifiée par méthode de test : CC-TST.2

Au premier échec, JUnit arrête la méthode : les valeurs suivantes ne sont plus vérifiées

Plusieurs assertions restent permises sur le résultat d'une seule action

```
@Test
void reserverUnePlaceEnLaisseNeuf()
        throws SalleCompleteException {
    Salle salle = new Salle(10);

    salle.reserver(1);

    assertEquals(9, salle.placesLibres(),
            "Une place reservee sur 10 doit en laisser 9");
}

// un autre test pour 5 places, un autre pour 10
```

Pour aller plus loin : @ParameterizedTest

Facultatif : une seule méthode, lancée une fois par cas

Chaque cas apparaît comme un test distinct dans les résultats

@CsvSource donne à chaque cas son entrée et son attendu

```
@ParameterizedTest
@CsvSource({"1, 9", "5, 5", "10, 0"})
void reserverDiminueLesPlacesLibres(int places, int attendu)
        throws SalleCompleteException {
    Salle salle = new Salle(10);

    salle.reserver(places);

    assertEquals(attendu, salle.placesLibres(),
            "Le nombre de places libres doit diminuer d'autant");
}
```

Chaque test repart de zéro

JUnit crée un nouvel objet de test pour chaque méthode

Aucun test ne compte sur un autre, ni sur l'ordre : CC-TST.3

@BeforeEach regroupe ce que tous les tests préparent

```
class SalleTest {

    private Salle salle;

    @BeforeEach
    void preparer() {
        salle = new Salle(10);
    }

    @Test
    void uneSalleNeuveEstEntierementLibre() {
        assertEquals(10, salle.placesLibres(),
                "Une salle neuve doit etre libre");
    }
}
```

Un test se justifie par le défaut qu'il attrape

Pour chaque test : quelle erreur le ferait passer au rouge ?

S'il n'y en a aucune, il ne protège de rien

Un test qui attrape les mêmes erreurs qu'un autre n'ajoute rien

```
// le defaut :  >=  au lieu de  >
if (places >= placesLibres()) {
    throw new SalleCompleteException(
            "Plus que " + placesLibres() + " places");
}

// un seul test passe au rouge :
// reserverToutesLesPlacesLibres
```

À l'examen : écrire, lancer, corriger

On vous donne une classe, et parfois la liste des cas

Vos tests peuvent révéler un défaut : c'est prévu

Corrigez alors la classe, jamais le test qui a raison

Exercices

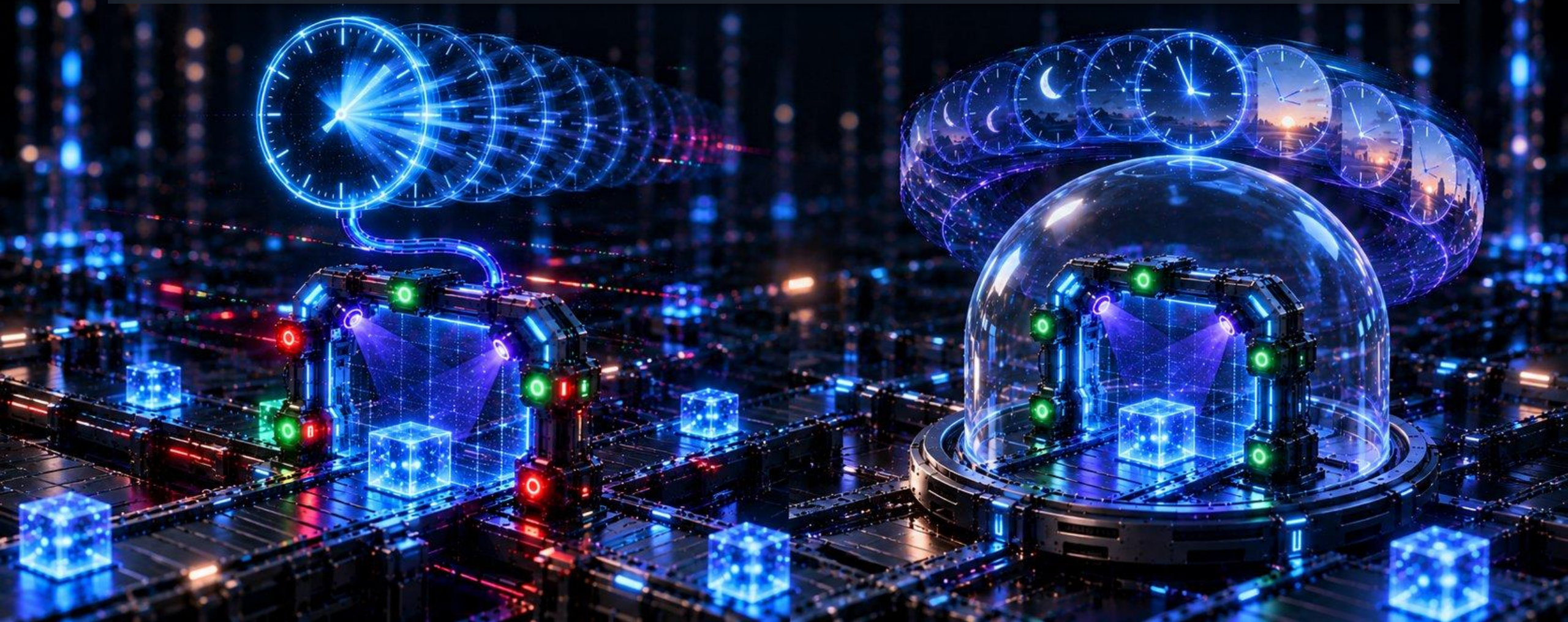


JUN.3 Écrire une classe de test

fascicule JUN

Le vert de `RedactionTest` ne dit rien de vos tests
Tests : `CC-TST.1`, `CC-TST.2`, `CC-TST.3`

5. Du code qui se laisse tester



Pourquoi certaines classes résistent aux tests

Un test prépare une situation, déclenche une action, vérifie le résultat : seul, et en un instant

Certaines classes l'en empêchent : elles ouvrent une fenêtre, lisent la date du jour, ou tirent au hasard

Le remède est dans la classe elle-même : une interface à la place de la fenêtre, un paramètre à la place de la date ou du hasard

Rappel : le contrôleur du module ITF

Au module ITF, le contrôleur ne dépend plus que d'une interface, Afficheur, reçue par setAfficheur

C'est ce qui va permettre de le tester sans rien ouvrir

```
public interface Afficheur {
    void affiche(String message);
}
public class Controleur {
    private Afficheur afficheur;
    public void setAfficheur(Afficheur afficheur) { this.afficheur = afficheur; }
    public void annoncer(String evenement) {
        afficheur.affiche("Evenement : " + evenement);
    }
}
```

Une doublure à la place de la fenêtre

Pour le test, on écrit un faux afficheur : il n'affiche rien, il retient le dernier message

Ce remplaçant s'appelle une doublure ; il vit dans `src/test`, à côté du test

```
class AfficheurQuiRetient implements Afficheur {

    String dernier;

    @Override
    public void affiche(String message) {
        dernier = message;
    }
}
```

Tester sans ouvrir la fenêtre

Le test branche la doublure, déclenche l'action, puis lit ce qu'elle a retenu

Aucune fenêtre ne s'ouvre : le test tourne en quelques millisecondes

```java
@Test
void leControleurSignaleLEvenement() {
    Controleur controleur = new Controleur();
    AfficheurQuiRetient afficheur = new AfficheurQuiRetient();
    controleur.setAfficheur(afficheur);

    controleur.annoncer("bouton presse");

    assertEquals("Evenement : bouton presse", afficheur.dernier);
}
```

Rappel : l'âge du livre, au module POO

Au module POO, l'âge d'un livre n'était pas stocké : `getAge` le calculait à partir de la date du jour

Juste, mais difficile à tester : le résultat change chaque année, sans que le code change

```
public class Livre {
    private String titre;
    private int anneeDeParution;

    public int getAge() {
        return LocalDate.now().getYear() - anneeDeParution;
    }
}
```

Ni le jour, ni le hasard

Le calcul reçoit l'année en paramètre : le test la fixe, et le résultat ne bouge plus

Une version de confort, sans paramètre, va chercher l'année et délègue au calcul

```
public int getAge(int anneeCourante) {          // testable
    return anneeCourante - anneeDeParution;
}
public int getAge() {                           // pratique
    return getAge(LocalDate.now().getYear());
}

assertEquals(61, dune.getAge(2026));   // vrai aujourd'hui, et dans dix ans
```